

Spec-Driven Development with Supervised RAG Refinement

AI-Assisted Software Development for Operational Weather Forecasting

NOAA EMC Global Workflow MCP Team

November 19, 2025

Section 1

The Problem

Operational Weather Forecasting at Scale

NOAA Global Workflow (GFS/GEFS) -
500K LOC Fortran (UFS model) - 300K LOC
C/Fortran (GSI/GDAS) - 2,000+ shell scripts -
7,000+ Rocoto task dependencies - 5 HPC
platforms

Challenges - Few understand complete system
- Documentation decays faster than code - 6-12
month onboarding - Manual compliance
auditing - Silent failures cascade

System Complexity

The Core Issue: Domain expertise is scarce,
documentation complexity exceeds human
cognitive capacity

Section 2

The SDD Framework Solution

Three Core Innovations

1. Machine-Readable Specs

- YAML/Markdown workflows
 - Parseable by AI agents
 - Systematic validation
 - Automated execution
- ✓ Eliminates ambiguity

2. Hybrid Graph-RAG

- Vector embeddings (semantic)
 - Neo4j graph (structure)
 - 78K+ code relationships
 - Intent-aware search
- ✓ 3.8× retrieval accuracy

3. SME Refinement

- Semantic annotations
 - Structured review guide
 - Domain expert validation
 - Continuous feedback
- ✓ 93% precision improvement

Section 3

Semantic Annotations: The “Aha!” Moment

Text Search vs. Intent-Aware Search

Traditional Text Search

Query: "How do I check for errors?"

Results: (hundreds of hits)

- "error message format"
- "error handling overview"
- "error codes reference"
- ...

Problems: - No priority information - No operational context - No relationship info - 45% precision

Intent-Aware Search (SDD)

```
.. mcp:intent:: rapid_error_detection
   :description: Enable immediate detection
   :rationale: 99% on-time delivery SLA
   :enforcement: runtime_check
```

```
.. mcp:utility:: err_chk
   :module: prod_util
   :priority: critical
   :required: yes
```

Benefits: - Structured metadata - Operational rationale - Clear relationships - **87% precision**

Section 4

MCP Directive System

7 Directive Types for Complete Semantic Coverage

Directive	Purpose	Key Insight
mcp:compliance	Priority/scope	Critical vs. recommended
mcp:intent	Why it exists	Operational rationale
mcp:severity	RFC 2119 level	must/should/may
mcp:utility	Tool metadata	Module, category, lifecycle
mcp:example	Code examples	Context-aware patterns
mcp:pattern	Design patterns	Recognize good/bad code
mcp:see-also	Relationships	Prerequisite/alternative

Innovation: Embeds in RST documentation → renders normally for humans, processable by AI

Section 5

SME Review Guide: Supervised Refinement

7-Dimension Expert Review Methodology

- ➊ **Intent Accuracy:** Does ':rationale:' capture true operational purpose?
- ➋ **Priority/Severity:** Does priority reflect real operational impact?
- ➌ **Utility Metadata:** Are module names, categories correct?
- ➍ **Code Examples:** Do examples show the RIGHT way?
- ➎ **Relationships:** Are ':related:' items truly connected?
- ➏ **Patterns:** Should this pattern be recognized/flagged?
- ➐ **Gap Analysis:** What's missing?

Workflow: Pilot → SME Review → Iterate → Validate → Deploy

Result: Transform “finds text” → “understands intent”

Section 6

SME Review Impact: The Numbers

Measured Improvements in NOAA Global Workflow

Metric	Before	After	Improvement
Retrieval Precision	45%	87%	+93%
False Positives (Compliance)	35%	8%	-77%
Query Resolution Time	2-5 min	15-30 sec	-80%
New Developer Onboarding	8 weeks	3 weeks	-63%

Key Finding: Domain expert refinement transforms RAG from “keyword matcher” to “intent reasoner”

Section 7

Hybrid RAG Architecture

Vector + Graph + Annotations = Magic

Pure Vector Search (Baseline) - ChromaDB with 768-dim embeddings - Semantic similarity only - **P@10: 0.45**

+ **Graph Context** - Neo4j: 78,339 relationships - CALLS, IMPORTS, DEFINES - **P@10: 0.67**

+ **Semantic Annotations** - MCP directives with intent - Priority, rationale, patterns - **P@10: 0.87** (3.8× improvement)

Ranking Function:

$$\text{score} = \begin{cases} \alpha \cdot \text{cosine} \\ + \beta \cdot \text{graph} \\ + \gamma \cdot \text{annot} \end{cases}$$

Weights (default): - $\alpha = 0.5$ (semantic) - $\beta = 0.3$ (structure) - $\gamma = 0.2$ (intent)

Query-Type Adaptive

Section 8

Query Type Optimization

Different Questions Need Different Approaches

Query Type	Vector	Graph	Annot	Example
Concept	0.7	0.1	0.2	"What is Rocoto?"
Usage	0.5	0.3	0.2	"How do I use err_chk?"
Code Structure	0.2	0.7	0.1	"What calls this function?"
Compliance	0.3	0.2	0.5	"Is this EE2 compliant?"
Troubleshoot	0.4	0.4	0.2	"Why is this failing?"

System automatically detects query type and adjusts weights

Graph dominates for structure queries, annotations for compliance, vectors for concepts

Section 9

Retrieval Accuracy Evaluation

100 Developer Queries, Expert-Labeled Ground Truth

Approach	Precision@10	MRR	Recall@50
Text Search (Baseline)	0.23	0.31	0.48
Pure Vector (MPNet)	0.45	0.52	0.71
Vector + Annotations	0.67	0.73	0.84
Hybrid (All)	0.87	0.91	0.95

Query Category Breakdown: Hybrid wins 90% of usage queries, 95% of structure queries, 93% of compliance checks

Section 10

Development Velocity: Real Task Comparison

Before SDD (Estimated)

- Manual code archaeology: **4-6 hours**
 - grep through 2,800 files
 - trace variable definitions
 - understand context
- Documentation writing: **2-3 hours**
 - synthesize findings
 - write explanations
- Review cycles: **2-4 hours**
 - back-and-forth clarifications

Total: 8-13 hours

With SDD (Actual)

- RAG-assisted analysis: **45 minutes**
 - hybrid query: instant context
 - graph traversal: call chains
 - annotations: intent
- Spec-driven docs: **30 minutes**
 - template-based generation
 - validated examples
- Automated validation: **15 minutes**
 - health checks pass

Total: 1.5 hours

5.3-8.7× faster with higher accuracy

Section 11

YAML Task Tracking

Complex Multi-Week Projects as Code

EVS EE2 Compliance Remediation Example:

metadata:

```
name: evs_ee2_compliance_remediation
duration: "12 weeks"
effort: "462 hours"
files_impacted: 670
```

phases:

- phase_1_critical_path: *# Weeks 1-3, 32 P0 scripts*
- phase_2_secondary: *# Weeks 4-6, 95 P1 scripts*
- phase_3_bulk: *# Weeks 7-10, 562 remaining*
- phase_4_validation: *# Weeks 11-12, integration*

resources:

```
personnel: 5.25 FTE-months
```


Section 12

Technical Insights: Lessons Learned

What Works, What Doesn't

Graph Enrichment Non-Negotiable

Pure vector fails for code intelligence.
Structure matters.

Quality > Quantity

10 SME-reviewed sections beat 100
auto-generated annotations.

MPNet Sweet Spot

768-dim provides best quality/speed/cost
(25ms inference, P@10=0.87).

Health Monitoring First-Class

Integrated from initialization, not bolted on
later.

Don't Auto-Generate Without SME Review

Garbage annotations → garbage retrieval.

Don't Ignore Version Control

Stale embeddings cause silent failures.
CI/CD pipeline essential.

Don't Skip Developer Training

Show 3-5× improvements on real tasks to
build trust.

Don't Attempt Big-Bang Rollout

Pilot → validate → scale with proven
patterns.

Section 13

Operational Deployment Architecture

Production-Ready Infrastructure

Knowledge Base Coverage - 5,307

documents ingested - 78,339 code relationships
- 2,800+ files across 7 repos - EE2 standards (complete) - CI test case docs

Services (Systemd-Managed) - ChromaDB (Docker, port 8080) - Neo4j (Docker, ports 7474/7687) - MCP Server (Node.js v4.0.0) - VS Code/Cursor integration

HPC Platforms - Hera, Hercules, Orion (RDHPCS) - WCOS2 (NCEP Operations) - 8-core VM, 32GB RAM - 100GB knowledge base storage

CI/CD Pipeline - Auto-trigger on code commits - Incremental graph updates - Re-generate affected embeddings - Validate retrieval accuracy

Section 14

The SDD Paradigm Shift

From “Find Text” to “Understand Intent”

Before: Developer asks → AI searches keywords → Returns generic text

⇓ (*Transform with SDD Framework*)

After: Developer asks → AI understands intent → Returns operational context

Key Innovation: SME annotations capture the *why* behind the *what*

Result: AI reasons like a domain expert, not a keyword matcher

Section 15

Future Work

Roadmap for Enhanced Capabilities

- ❶ **Multi-Modal Embeddings:** Joint code-text embeddings (understand code semantics directly)
- ❷ **Temporal Query Analysis:** Track poorly-answered queries → prioritize annotation efforts
- ❸ **Cross-Repository Intelligence:** Unified graph across Global Workflow, UFS, GSI, wxflow
- ❹ **Real-Time Collaboration:** Multi-developer annotation editing interface
- ❺ **Automated Annotation Generation:** LLM generates initial annotations → SME validates

Section 16

Conclusion

Three Innovations, One Framework

- ① **Machine-Readable Specs:** Workflows as code → systematic execution
- ② **Semantic Annotations:** Human-readable + machine-processable
- ③ **SME Refinement:** Structured methodology for expert knowledge capture

Empirical Validation (NOAA Global Workflow): - $3.8\times$ retrieval accuracy (P@10: 0.23 → 0.87) - $5\text{--}9\times$ development velocity (13 hours → 1.5 hours) - 77% false positive reduction (35% → 8%)

Key Takeaway: AI assistance succeeds when domain experts guide the knowledge base through structured methodologies.

Pure automation fails. Supervised human-AI collaboration succeeds.

Section 17

Questions?

Thank You!

Contact:

NOAA EMC Global Workflow MCP Team

Terry.McGuinness@noaa.gov

Resources:

Framework Specs: